

✓ Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
```

```
# Pre-Requirment
warnings.filterwarnings(action='ignore')
```

✓ Load Data into DataFrame

```
df = pd.read_csv('dataset.csv')
```

✓ Data cleaning:

```
df.head()
```

	policy_id	policy_tenure	age_of_car	age_of_policyholder	area_cluster	popu
0	ID00001	0.515874	0.05	0.644231	C1	
1	ID00002	0.672619	0.02	0.375000	C2	
2	ID00003	0.841110	0.02	0.384615	C3	
3	ID00004	0.900277	0.11	0.432692	C4	
4	ID00005	0.596403	0.11	0.634615	C5	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30100 entries, 0 to 30099
Data columns (total 44 columns):
#   Column                                Non-Null Count  Dtype
---  -

```

```

0  policy_id          30100 non-null object
1  policy_tenure      30100 non-null float64
2  age_of_car         30100 non-null float64
3  age_of_policyholder 30100 non-null float64
4  area_cluster       30100 non-null object
5  population_density 30100 non-null object
6  make              30100 non-null int64
7  is_claim          30100 non-null int64
8  segment           30100 non-null object
9  model             30100 non-null object
10 fuel_type         30100 non-null object
11 max_torque        30100 non-null object
12 max_power         30100 non-null object
13 engine_type       30100 non-null object
14 airbags           30100 non-null int64
15 is_esc            30100 non-null object
16 is_adjustable_steering 30100 non-null object
17 is_tpms           30100 non-null object
18 is_parking_sensors 30100 non-null object
19 is_parking_camera 30100 non-null object
20 rear_brakes_type  30100 non-null object
21 displacement      30100 non-null int64
22 cylinder          30100 non-null int64
23 transmission_type 30100 non-null object
24 gear_box          30100 non-null int64
25 steering_type     30100 non-null object
26 turning_radius    30100 non-null float64
27 Length            30100 non-null object
28 width             30100 non-null object
29 height            30100 non-null int64
30 Gross_weight      30100 non-null object
31 is_front_fog_lights 30100 non-null object
32 is_rear_window_wiper 30100 non-null object
33 is_rear_window_washer 30100 non-null object
34 is_rear_window_defogger 30100 non-null object
35 is_brake_assist    30100 non-null object
36 is_power_door_locks 30100 non-null object
37 is_central_locking 30100 non-null object
38 is_power_steering 30100 non-null object
39 is_driver_seat_height_adjustable 30100 non-null object
40 is_day_night_rear_view_mirror 30100 non-null object
41 is_ecw            30100 non-null object
42 is_speed_alert     30100 non-null object
43 ncap_rating        30100 non-null int64
dtypes: float64(4), int64(8), object(32)
memory usage: 10.1+ MB

```

```
df.describe()
```

	policy_tenure	age_of_car	age_of_policyholder	make	is_cla
count	30100.000000	30100.000000	30100.000000	30100.000000	30100.0000
mean	0.612097	0.069436	0.469028	1.758272	0.0640
std	0.414670	0.056833	0.122673	1.136681	0.2449
min	0.002739	0.000000	0.288462	1.000000	0.0000
25%	0.207099	0.020000	0.365385	1.000000	0.0000
50%	0.575252	0.060000	0.451923	1.000000	0.0000
75%	1.040013	0.110000	0.548077	3.000000	0.0000
max	1.376478	1.000000	0.971154	5.000000	1.0000

```
df.isna().sum()
```

```

policy_id          0
policy_tenure      0
age_of_car         0
age_of_policyholder 0
area_cluster       0
population_density 0
make               0
is_claim           0
segment            0
model              0
fuel_type          0
max_torque         0
max_power          0
engine_type        0
airbags            0
is_esc             0
is_adjustable_steering 0
is_tpms            0
is_parking_sensors 0
is_parking_camera  0
rear_brakes_type   0
displacement       0
cylinder           0
transmission_type  0
gear_box           0
steering_type      0
turning_radius     0
Length             0
width              0
height             0
Gross_weight       0
is_front_fog_lights 0
is_rear_window_wiper 0
is_rear_window_washer 0

```

```

is_rear_window_defogger      0
is_brake_assist              0
is_power_door_locks         0
is_central_locking           0
is_power_steering            0
is_driver_seat_height_adjustable 0
is_day_night_rear_view_mirror 0
is_ecw                       0
is_speed_alert               0
ncap_rating                  0
dtype: int64

```

```
df.duplicated().sum()
```

```
np.int64(0)
```

✓ Remove Unwanted Columns

```
df.drop(['policy_id'],axis=1,inplace=True)
```

✓ Convert Object into numerical values

```

df["population_density"] = pd.to_numeric(df["population_density"],errors='coerc
df.head()

```

	policy_tenure	age_of_car	age_of_policyholder	area_cluster	population_dens
0	0.515874	0.05	0.644231	C1	495
1	0.672619	0.02	0.375000	C2	1
2	0.841110	0.02	0.384615	C3	40
3	0.900277	0.11	0.432692	C4	1
4	0.596403	0.11	0.634615	C5	347

```
df["population_density"].isna().sum()
```

```
np.int64(15)
```

```

df["population_density"].fillna(df['population_density'].median(),inplace = Tru
df["population_density"].isna().sum()

```

```
np.int64(0)
```

```
df.head()
```

	policy_tenure	age_of_car	age_of_policyholder	area_cluster	population_dens
0	0.515874	0.05	0.644231	C1	499
1	0.672619	0.02	0.375000	C2	879
2	0.841110	0.02	0.384615	C3	409
3	0.900277	0.11	0.432692	C4	879
4	0.596403	0.11	0.634615	C5	3479

```
df=pd.get_dummies(df,columns=['segment','model','fuel_type','engine_type','rear
```

```
df.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30100 entries, 0 to 30099
Data columns (total 74 columns):
Column Non-Null Count Dtype
--- -
0 policy_tenure 30100 non-null float64
1 age_of_car 30100 non-null float64
2 age_of_policyholder 30100 non-null float64
3 area_cluster 30100 non-null object
4 population_density 30100 non-null float64
5 make 30100 non-null int64
6 is_claim 30100 non-null int64
7 max_torque 30100 non-null object
8 max_power 30100 non-null object
9 airbags 30100 non-null int64
10 is_esc 30100 non-null object
11 is_adjustable_steering 30100 non-null object
12 is_tpms 30100 non-null object
13 is_parking_sensors 30100 non-null object
14 is_parking_camera 30100 non-null object
15 displacement 30100 non-null int64
16 cylinder 30100 non-null int64
17 gear_box 30100 non-null int64
18 turning_radius 30100 non-null float64
19 Length 30100 non-null object
20 width 30100 non-null object
21 height 30100 non-null int64
22 Gross_weight 30100 non-null object

```

23 is_front_fog_lights          30100 non-null object
24 is_rear_window_wiper        30100 non-null object
25 is_rear_window_washer       30100 non-null object
26 is_rear_window_defogger     30100 non-null object
27 is_brake_assist              30100 non-null object
28 is_power_door_locks         30100 non-null object
29 is_central_locking           30100 non-null object
30 is_power_steering            30100 non-null object
31 is_driver_seat_height_adjustable 30100 non-null object
32 is_day_night_rear_view_mirror 30100 non-null object
33 is_ecw                       30100 non-null object
34 is_speed_alert               30100 non-null object
35 ncap_rating                  30100 non-null int64
36 segment_A                    30100 non-null int64
37 segment_B1                   30100 non-null int64
38 segment_B2                   30100 non-null int64
39 segment_C1                   30100 non-null int64
40 segment_C2                   30100 non-null int64
41 segment_Utility              30100 non-null int64
42 model_M1                     30100 non-null int64
43 model_M10                    30100 non-null int64
44 model_M11                    30100 non-null int64
45 model_M2                     30100 non-null int64
46 model_M3                     30100 non-null int64
47 model_M4                     30100 non-null int64
48 model_M5                     30100 non-null int64
49 model_M6                     30100 non-null int64
50 model_M7                     30100 non-null int64
51 model_M8                     30100 non-null int64
52 model_M9                     30100 non-null int64

```

```

#To view all columns of DataFrame
pd.set_option('display.max_columns', None)
df.head()

```

	policy_tenure	age_of_car	age_of_policyholder	area_cluster	population_dens
0	0.515874	0.05	0.644231	C1	495
1	0.672619	0.02	0.375000	C2	875
2	0.841110	0.02	0.384615	C3	405
3	0.900277	0.11	0.432692	C4	875
4	0.596403	0.11	0.634615	C5	3475

✓ Converting Categorical columns into numerical

```

from sklearn.preprocessing import OrdinalEncoder
cols = ['is_esc',
        'is_adjustable_steering',
        'is_tpms',
        'is_parking_sensors',
        'is_parking_camera',
        'is_front_fog_lights',
        'is_rear_window_wiper',
        'is_rear_window_washer',
        'is_rear_window_defogger',
        'is_brake_assist',
        'is_power_door_locks',
        'is_central_locking',
        'is_power_steering',
        'is_driver_seat_height_adjustable',
        'is_day_night_rear_view_mirror',
        'is_ecw',
        'is_speed_alert',
        'area_cluster'
]
df[cols] = OrdinalEncoder().fit_transform(df[cols])

```

```
df.head()
```

	policy_tenure	age_of_car	age_of_policyholder	area_cluster	population_dens
0	0.515874	0.05	0.644231	0.0	49%
1	0.672619	0.02	0.375000	11.0	87%
2	0.841110	0.02	0.384615	15.0	40%
3	0.900277	0.11	0.432692	16.0	87%
4	0.596403	0.11	0.634615	17.0	347%

✓ Split columns

```

# Splitting max_power column into power & power_rpm
df[['power', 'power_rpm']] = df['max_power'].str.extract(r'(\d+)\D+(\d+)')
df[['power', 'power_rpm']] = df[['power', 'power_rpm']].astype(float)

```

```
# Splitting max_torque column into torque & torque_rpm
df[['torque', 'rpm']] = df['max_torque'].str.extract(r'(\d+)\D+(\d+)')
df[['torque', 'rpm']] = df[['torque', 'rpm']].astype(float)
```

```
# extracting numerical values from Length and Width
df['Length'] = df['Length'].str.extract(r'(\d+)').astype(float)
df['width'] = df['width'].str.extract(r'(\d+)').astype(float)
```

```
#extracting numerical values from Gross_weight
df['Gross_weight'] = df['Gross_weight'].str.extract(r'(\d+)').astype(float)
```

```
df.drop(['max_torque', 'max_power'], axis=1, inplace = True)
```

```
df['torque/rpm'] = df['torque']/df['rpm']
df['power/rpm'] = df['power']/df['power_rpm']
```

```
len(df.columns)
```

```
78
```

```
df.head()
```

	policy_tenure	age_of_car	age_of_policyholder	area_cluster	population_dens
0	0.515874	0.05	0.644231	0.0	49%
1	0.672619	0.02	0.375000	11.0	87%
2	0.841110	0.02	0.384615	15.0	40%
3	0.900277	0.11	0.432692	16.0	87%
4	0.596403	0.11	0.634615	17.0	347%

✓ Scalling

```
from sklearn.preprocessing import MinMaxScaler
cols=['population_density', 'Length', 'width', 'height', 'Gross_weight', 'power', 'pc
df[cols] = MinMaxScaler().fit_transform(df[cols])
```

```
df.head()
```


	policy_tenure	age_of_car	age_of_policyholder	area_cluster	population_dens
0	0.515874	0.05	0.644231	0.0	0.064
1	0.672619	0.02	0.375000	11.0	0.116
2	0.841110	0.02	0.384615	15.0	0.051
3	0.900277	0.11	0.432692	16.0	0.116
4	0.596403	0.11	0.634615	17.0	0.470

```
len(df.columns)
```

```
78
```

✓ Save Cleaned Data

```
df.to_csv('CleanedDataset.csv', index=False)
```

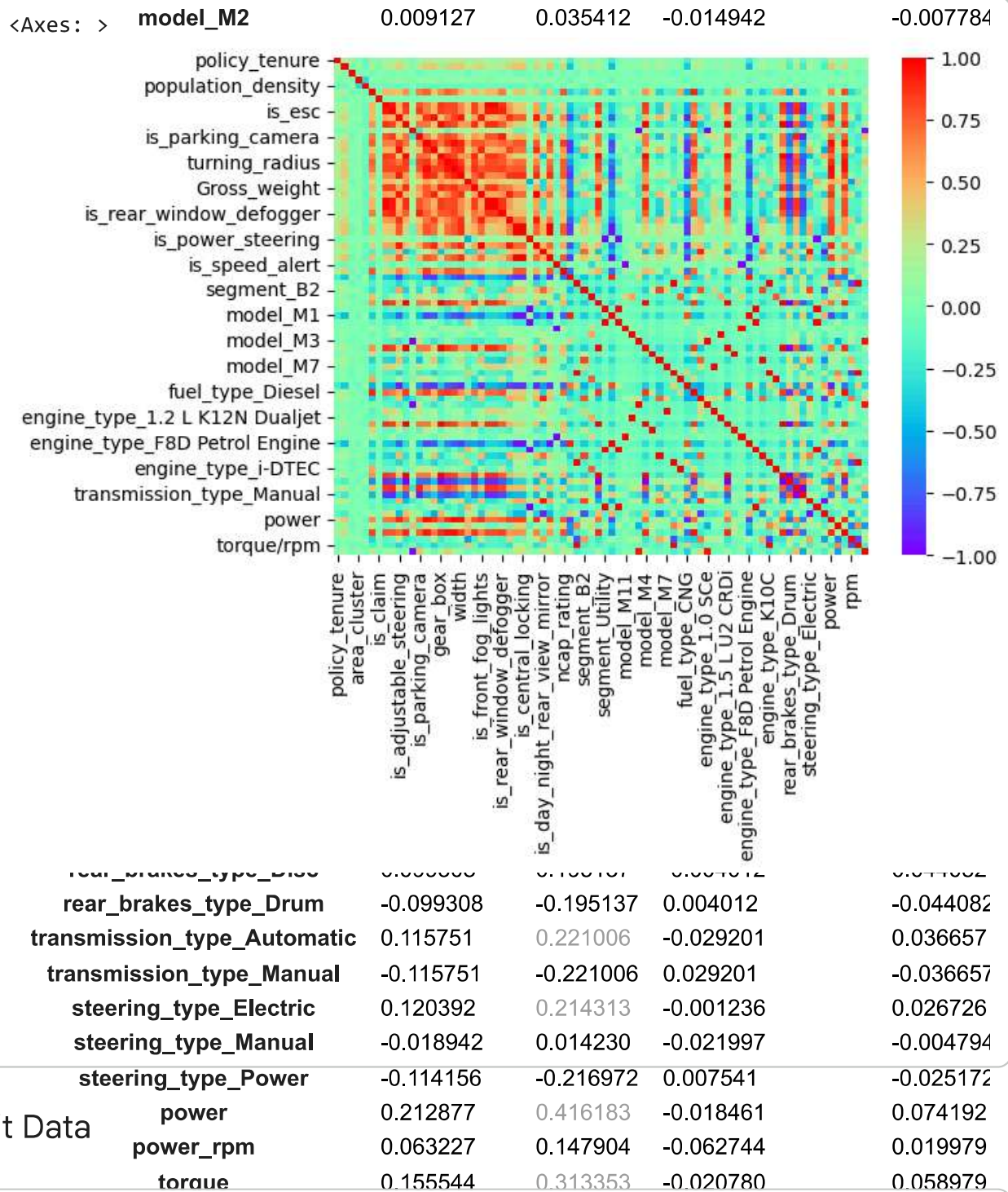
Start coding or [generate](#) with AI.

✓ EDA

```
pd.set_option('display.max_rows', None)
df.corr().style.background_gradient()
```



	policy_tenure	age_of_car	age_of_policyholder	area_cluster
policy_tenure	1.000000	0.171386	0.147808	0.029256
age_of_car	0.171386	1.000000	-0.029554	0.051299
age_of_policyholder	0.147808	-0.029554	1.000000	0.007507
area_cluster	0.029256	0.051299	0.007507	1.000000
population_density	-0.100394	-0.068776	0.011104	-0.319268
make	0.088044	0.191988	-0.039068	0.031889
is_claim	0.079154	-0.031418	0.019070	0.013325
airbags	0.109305	0.211192	-0.010552	0.042247
is_esc	0.110524	0.228586	-0.018830	0.040548
is_adjustable_steering	0.197950	0.378014	0.008124	0.066459
is_tpms	0.099308	0.195137	-0.004012	0.044082
is_parking_sensors	-0.023141	-0.017220	0.031712	0.003252
is_parking_camera	0.117971	0.230600	-0.045722	0.034825
displacement	0.198387	0.399077	-0.029888	0.069332
cylinder	0.194389	0.386019	0.001745	0.065723
gear_box	0.100030	0.202973	-0.006572	0.045296
turning_radius	0.174401	0.336290	-0.024294	0.062691
Length	0.196974	0.388762	-0.026302	0.069308
width	0.219017	0.421622	-0.013226	0.075892
height	0.116388	0.257032	-0.057061	0.047268
Gross_weight	0.144895	0.303680	-0.012059	0.060354
is_front_fog_lights	0.194399	0.357638	0.009567	0.064386
is_rear_window_wiper	0.108937	0.214479	-0.012390	0.042304
is_rear_window_washer	0.108937	0.214479	-0.012390	0.042304
is_rear_window_defogger	0.111140	0.228907	-0.033610	0.037005
is_brake_assist	0.190827	0.356867	0.021891	0.067607
is_power_door_locks	0.234621	0.440346	-0.047953	0.074240
is_central_locking	0.234621	0.440346	-0.047953	0.074240
is_power_steering	0.018942	-0.014230	0.021997	0.004794
is_driver_seat_height_adjustable	0.196219	0.366733	0.007333	0.065923
is_day_night_rear_view_mirror	0.120357	0.200239	0.000302	0.025653
is_ecw	0.234621	0.440346	-0.047953	0.074240
is_speed_alert	-0.008766	-0.051810	0.013996	-0.008705
ncap_rating	0.176408	0.352992	-0.040925	0.061837
segment_A	0.212781	0.428987	0.040945	0.072605
sns.heatmap(df.corr(), cmap = 'rainbow')				
segment_B2	0.111187	0.202508	0.032181	0.034400
segment_C1	0.015364	0.050360	-0.043538	-0.006299
segment_C2	0.099308	0.195137	-0.004012	0.044082
segment_Utility	-0.018942	0.014230	-0.021997	-0.004794
model_M1	-0.234322	-0.456002	0.056319	-0.074536
model_M10	-0.018942	0.014230	-0.021997	-0.004794
model_M11	0.008766	0.051810	-0.013996	0.008705



Split Data

```
x = df.drop(['make','is_claim'],axis=1)
y = df['is_claim']
```

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,train_size=0.8)
```

Smote

```
from imblearn.combine import SMOTETomek
smote_tomek = SMOTETomek(random_state=42)
x_train_sm, y_train_sm = smote_tomek.fit_resample(x_train, y_train)
```

✓ Functions Of All Models

✓ Logistic Regression

```
from sklearn.linear_model import LogisticRegressionCV

def logistic_model():
    model = LogisticRegressionCV(cv=5,max_iter=1000)
    model.fit(x_train_sm,y_train_sm)
    return model
```

✓ KNN

```
from sklearn.neighbors import KNeighborsClassifier

def knn_model() :
    model = KNeighborsClassifier(n_neighbors = 100)
    model.fit(x_train_sm,y_train_sm)
    return model
```

✓ SVM

```
from sklearn.svm import SVC

def svm_model():
    model = SVC()
    model.fit(x_train_sm,y_train_sm)
    return model
```

✓ Decision Tree

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
def decision_model():
    model = DecisionTreeClassifier()

    param = {
        "criterion" : ["gini","log_loss","entropy"],
        "max_depth" : [10,20,30,40,50,60]
```

```

    }

    grid_tree = GridSearchCV(model,param)
    grid_tree.fit(x_train_sm,y_train_sm)
    return grid_tree.best_estimator_

```

✓ Model Evaluation

```

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

def evaluate_model(model):
    y_pred = model.predict(x_test)
    testing_accuracy = accuracy_score(y_test, y_pred)
    testing_precision = precision_score(y_test, y_pred,average='weighted')
    testing_recall = recall_score(y_test, y_pred,average='weighted')
    testing_f1 = f1_score(y_test, y_pred,average='weighted')

    return testing_accuracy, testing_precision, testing_recall, testing_f1

```

✓ Generate Performance Chart

```

models = [
    ("Logistic Regression",logistic_model(),(0,0)),
    ("KNN",knn_model(),(0,1)),
    ("SVM",svm_model(),(1,0)),
    ("Decision Tree",decision_model(),(1,1))
]

```

```

performance_data = []
for model_name,model,position in models :
    test_accuracy,precision,recall,f1 = evaluate_model(model)
    performance_data.append([
        model_name,f"{test_accuracy*100:0.2f}%",f"{precision*100:0.2f}%",f"{recall*100:0.2f}%",f"{f1*100:0.2f}%"
    ])
performance_chart = pd.DataFrame(performance_data,columns=["Model","Test Accuracy","Precision","Recall","F1"])

```

	Model	Test Accuracy	Precision	Recall	F1
0	Logistic Regression	56.81%	89.60%	56.81%	67.41%